CREDITGUARD

MACHINE LEARNING REPORT

Machine Learning Model Implementation

1.1 Data Preprocessing

Before training the machine learning models, the dataset was preprocessed to remove unnecessary features and convert categorical variables into numerical values.

Feature Selection

The following columns were removed because they were either unique identifiers, constant values, or not useful for prediction:

- ID
- FLAG_MOBIL
- DAYS_BIRTH
- DAYS_EMPLOYED

```
df=df.drop(['ID','FLAG_MOBIL','DAYS_BIRTH','DAYS_EMPLOYED'],axis=1)
```

Label Encoding

Since machine learning algorithms require numerical input, categorical columns were converted into numerical values using **LabelEncoder**.

The encoded columns include:

- CODE_GENDER
- FLAG_OWN_CAR
- FLAG_OWN_REALTY
- NAME_INCOME_TYPE
- NAME_EDUCATION_TYPE
- NAME_FAMILY_STATUS
- NAME_HOUSING_TYPE
- OCCUPATION_TYPE
- STATUS

```
from sklearn.preprocessing import LabelEncoder
label = LabelEncoder()
df.CODE_GENDER=label.fit_transform(df.CODE_GENDER)
df.FLAG_OWN_CAR=label.fit_transform(df.FLAG_OWN_CAR)
df.FLAG_OWN_REALTY=label.fit_transform(df.FLAG_OWN_REALTY)
df.NAME_INCOME_TYPE=label.fit_transform(df.NAME_INCOME_TYPE)
df.NAME_EDUCATION_TYPE=label.fit_transform(df.NAME_EDUCATION_TYPE)
df.NAME_FAMILY_STATUS=label.fit_transform(df.NAME_FAMILY_STATUS)
df.NAME_HOUSING_TYPE=label.fit_transform(df.NAME_HOUSING_TYPE)
df.OCCUPATION_TYPE=label.fit_transform(df.OCCUPATION_TYPE)
df.STATUS=label.fit_transform(df.STATUS)
```

1.2 Feature and Target Selection

The predictor variables (X) and target variable (y) were separated.

- Independent Variables (X): All features except CREDIT_RISK and STATUS
- Dependent Variable (y): CREDIT_RISK

```
x=df.drop(['CREDIT_RISK','STATUS'],axis=1).values
y=df['CREDIT_RISK'].values
```

1.3 Train-Test Split

The dataset was divided into training and testing sets using an 80:20 ratio.

```
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest= train_test_split(x,y,test_size=0.2)
```

1.4 Logistic Regression

Logistic Regression is a supervised classification algorithm that predicts the probability of a binary outcome.

```
from sklearn.linear_model import LogisticRegression
modellogistic= LogisticRegression()
modellogistic.fit(xtrain,ytrain)
yplogistic= modellogistic.predict(xtest)
```

- Logistic Regression achieved a **high classification accuracy of 98.96%** on the test dataset.
- The model is simple, computationally efficient, and easy to interpret.
- It showed **consistent performance** during 15-fold cross-validation with a mean accuracy of **98.95%**.
- The close agreement between testing accuracy and cross-validation accuracy indicates that the model is stable and generalizes well to unseen data.

1.5 Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy.

```
from sklearn.ensemble import RandomForestClassifier
modelR=RandomForestClassifier()
modelR.fit(xtrain,ytrain)
ypr=modelR.predict(xtest)
```

- Hyperparameter Tuning using GridSearchCV for Random Forest Classifier

To improve the Random Forest model, GridSearchCV was used.

```
from sklearn.model_selection import GridSearchCV
```

```
tuned_parameters = {'n_estimators': [50, 100, 200, 300, 500]}
model_gscv = RandomForestClassifier(random_state=42)
new_model = GridSearchCV(estimator=model_gscv,param_grid=tuned_parameters,scoring='accuracy',cv=15)
```

- Random Forest achieved a **high testing accuracy of 98.90%**.
- The model successfully identified both majority and minority class instances.
- It obtained an **F1-score of 0.216**, demonstrating its capability to detect positive (credit-risk) cases.
- The model achieved a **15-fold cross-validation accuracy of 97.31%**, indicating reliable performance across different subsets of the data.
- Random Forest effectively handles complex relationships between features and reduces overfitting through ensemble learning.

- GridSearchCV automatically selected the optimal number of trees for the Random Forest model.
- The best parameter obtained was **n_estimators = 500**.
- The tuned model achieved a **cross-validation accuracy of 98.81%**.
- Hyperparameter tuning improved the model's robustness and optimized its predictive performance.
- Using 500 decision trees increased the stability and reliability of the Random Forest classifier.

1.6 XGBoost Classifier

Extreme Gradient Boosting (XGBoost) is an advanced ensemble algorithm that builds decision trees sequentially to improve prediction performance.

```
from xgboost import XGBClassifier
modelxgc= XGBClassifier()
modelxgc.fit(xtrain,ytrain)
ypxgc= modelxgc.predict(xtest)
```

- XGBoost achieved the **highest testing accuracy of 98.97%** among all the evaluated models.
- The model obtained a **15-fold cross-validation accuracy of 98.36%**, indicating strong generalization ability.
- XGBoost is an advanced ensemble algorithm that efficiently captures complex patterns in the dataset.
- The model provides fast and accurate predictions while maintaining excellent overall classification performance.
- Its high accuracy demonstrates its effectiveness in handling large-scale classification problems.

5.11 Model Comparison

Model	Test Accuracy (%)	Cross Validation Accuracy (%)	F1 Score
Logistic Regression	98.96	98.95	0.000
Random Forest	98.90	97.31	0.216
XGBoost	98.97	98.36	0.157

Three machine learning models—**Logistic Regression, Random Forest, and XGBoost**—were implemented and evaluated for credit risk prediction. All models achieved **more than 98% accuracy**, demonstrating strong predictive performance. Logistic Regression provided a reliable baseline with consistent results, while Random Forest improved prediction through ensemble learning and further benefited from hyperparameter tuning using GridSearchCV. **XGBoost achieved the highest accuracy of 98.97%**, making it the best-performing model. Overall, the results indicate that machine learning techniques can effectively predict credit risk and support better decision-making in financial institutions.